Saminur Karim

Performing Web Application and Database Attacks

INTRODUCTION

Many websites today use JavaScript, it is usually enabled by default on most browsers. When a website allows JavaScript injected by an attacker to be executed in the context of one of its pages, it can lead to undesirable consequences. The attacker may be able to run JavaScript on their own machine to determine sensitive information. The attacker may also be able to control code that is executed on the machines of other clients visiting the same site.One goal of an ethical hacker is to find weaknesses in web applications like SQL injections, command execution attacks, and Javascript-based attacks. Penetration for web applications include phases like:

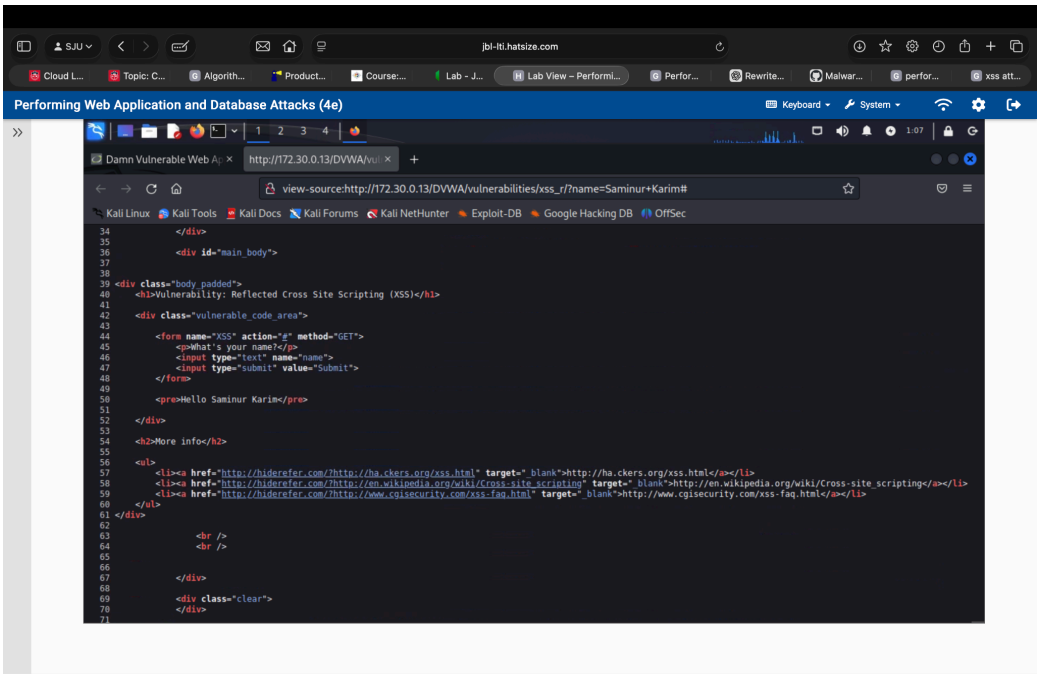
- Enumerating the pages that are accessible
- Identifying pages or API endpoints that may be vulnerable
- Attempting to exploit endpoints identified as possible attack vectors

In this lab, penetration testing is applied to vulnerable websites like the Damn Vulnerable Web Application (DVWA) and OWASP Juice Shop.

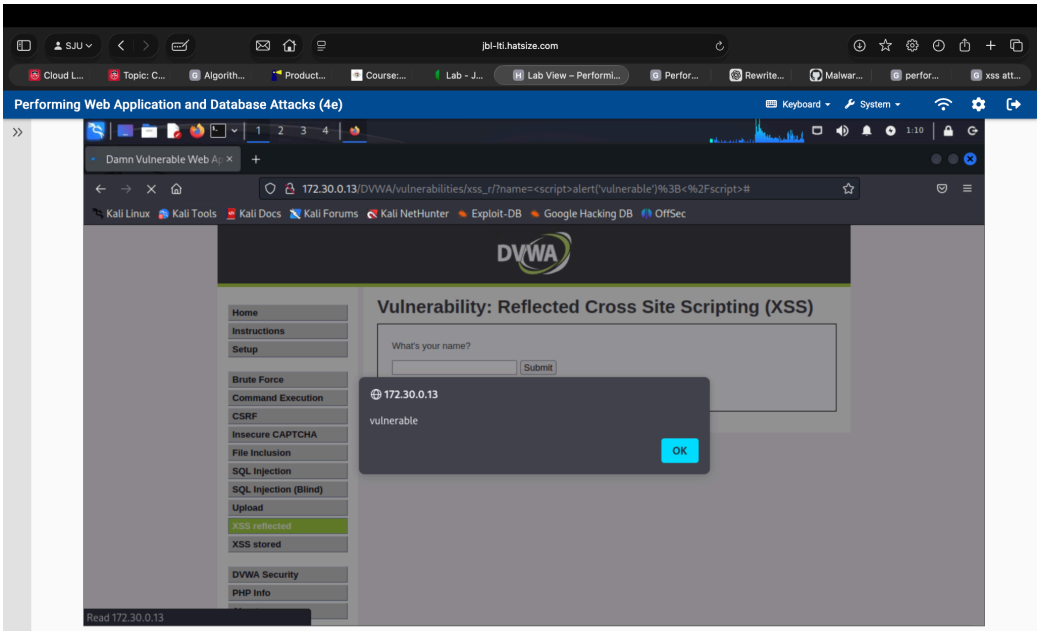
XSS ATTACK

One of the most common web application attacks is cross site scripting (XSS). XSS attacks injects malicious scripts into trusted sites, that are then executed on the victim's machine. There are 3 forms of XSS attacks: DOM-based XSS attack, reflected XSS attack, and persistent XSS attack. To perform this attack, the endpoint must accept and interpret input from the attacker. This can occur by editing the URL or inputting content into forms.

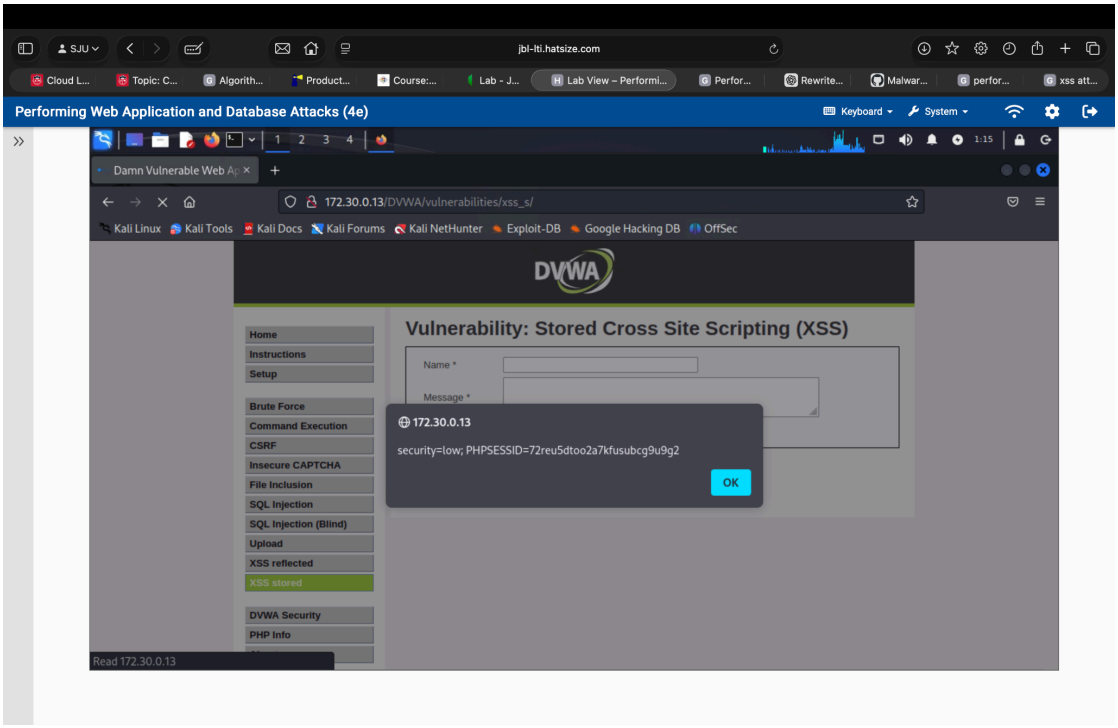
First, I entered the DVWA website and logged in. I navigated to DWVA security and changed the security level to low. Then, I clicked on the XSS reflected tab and input my name. I double clicked an empty spot and clicked on the view page source which gave access to the website's source HTML. I was able to view my name that I inputted previously.



In the screenshot below, I closed the source HTML page and inputted a script into the name field. The text was executable Javascript that returned the word “vulnerable”. This demonstrated a reflected XSS attack. The form submitted the text in the input field to the server, and the server returned a page that included the exact text from the form’s input field to the Firefox client. The text was expected to be displayed, but instead the server generated the page that included JavaScript that I controlled. The malicious code reflected from the client application and executed.

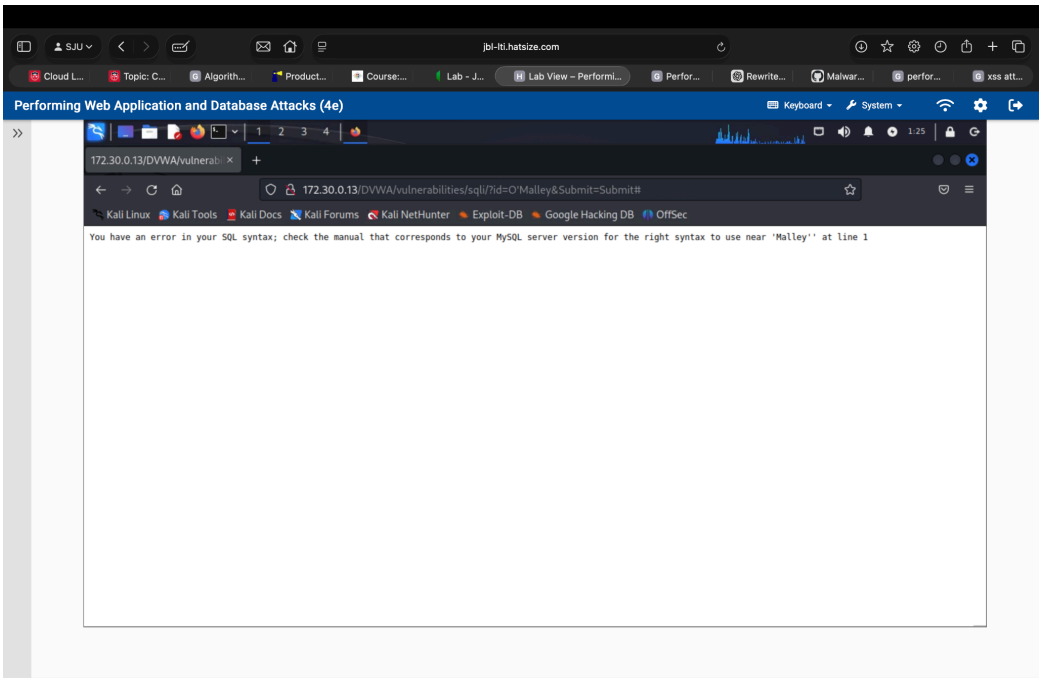


A persistent XSS attack is allowed to be executed repeatedly. Next, I clicked the XSS stored section. I input my name into the guestbookand another script to return a session ID and the cookies for security level. I clicked away from the XSS stored button by clicking home and returned back. Everytime someone navigates to the guestbook, their cookie information will be revealed.

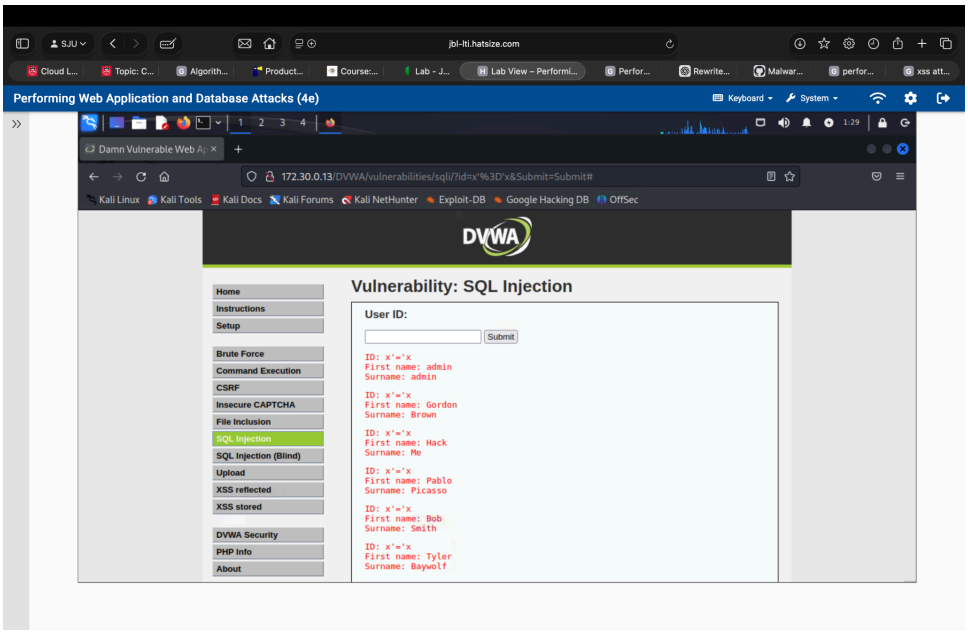


SQL INJECTION

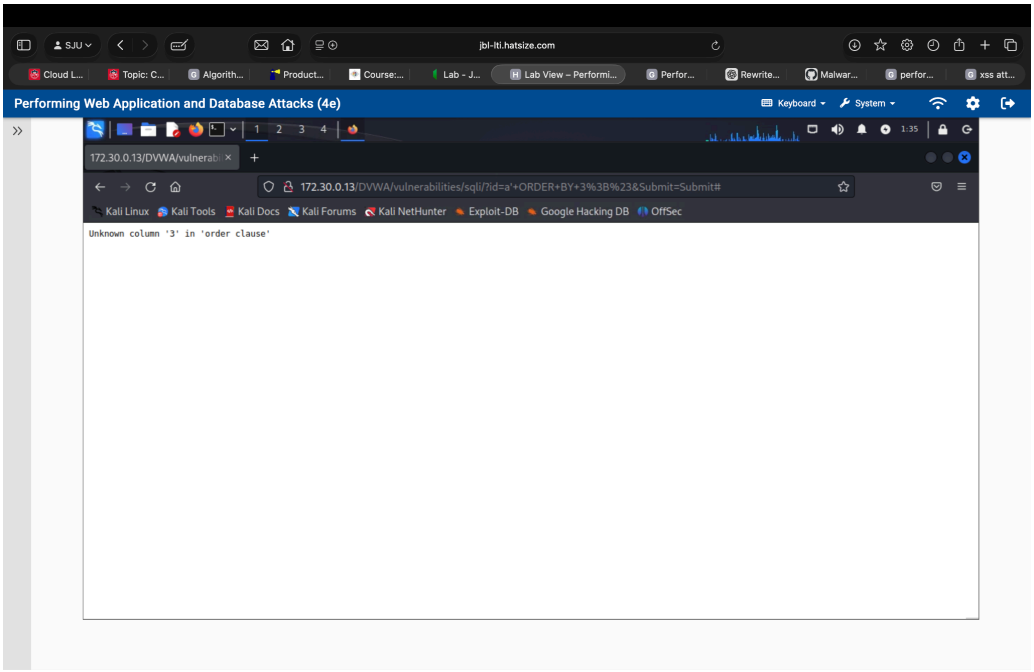
An SQL injection attack exploits the database using SQL statements. Attackers can manipulate, delete, and retrieve any data from the database using SQL injection. SQL injections usually occur when user input is part of the SQL that is executed. I navigated to the SQL injection in the sidebar. I entered the input "O'Malley" and an error page returned. Typically, an application should never return specific error pages. Applications should handle SQL syntax internally and return an error message that doesn't reveal sensitive information.



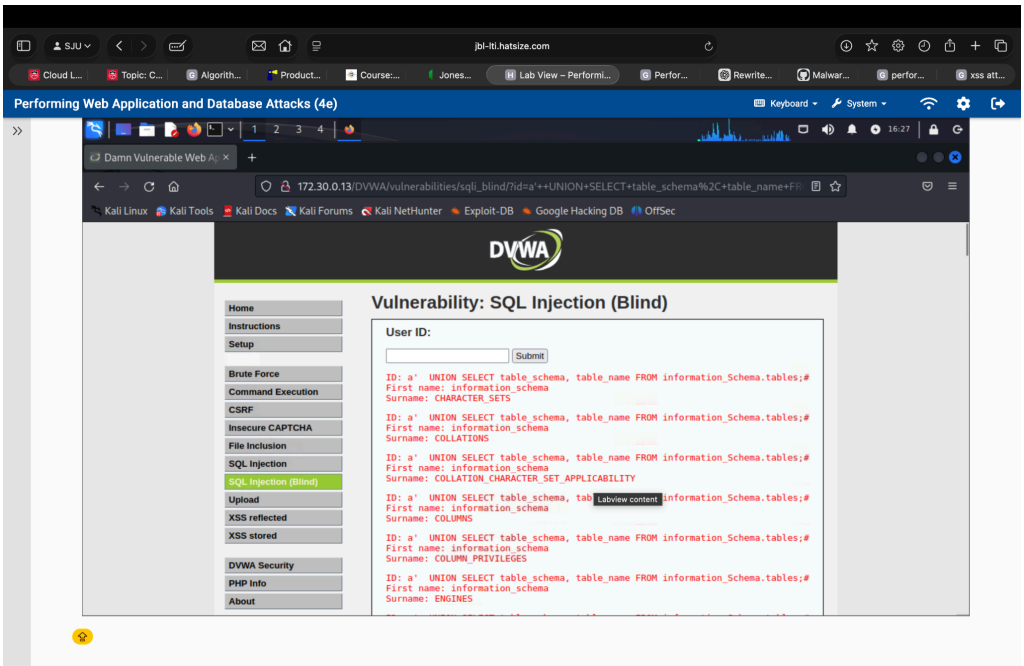
Syntax errors do not immediately mean an application can be exploited. I entered "name" into the user ID field. Nothing changed and the input field was empty again. I entered "1" and that returned one user record which is typical for a query. This is how attackers learn to manipulate applications for a vulnerability. I entered "x'='x" next and the query returned multiple records of user names and surnames.



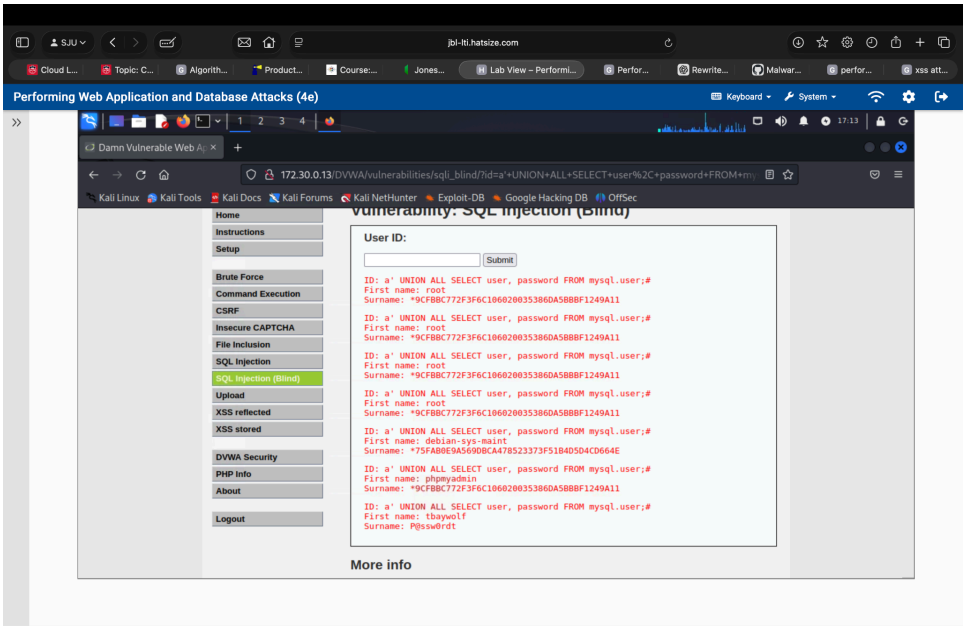
I continued with manipulating the input field. I changed the input to “a’ ORDER BY 1;#” and “a’ ORDER BY 2;#” which both refreshed the page back to normal. I then input “a’ ORDER BY 3;#” and that returned an error page.



Next, I input “a’ OR first_name IS NULL ;#” which emptied the field after being entered. This must mean there is a first_name column in a table used within the query. This is a one time method used to determine columns using SQL injection. Then, I input another query into the field to retrieve information from the SQL database. Using “a’ UNION SELECT table_scema, table_name FROM information_schema.tables;#” I began a query for User ID, surname, and first name. The SQL injection begins with the letter *a* followed by an apostrophe to provide an expected value for the User ID input field and terminate the original query string. By using UNION SELECT, you can attempt to append any two-column query to the results of the original query.

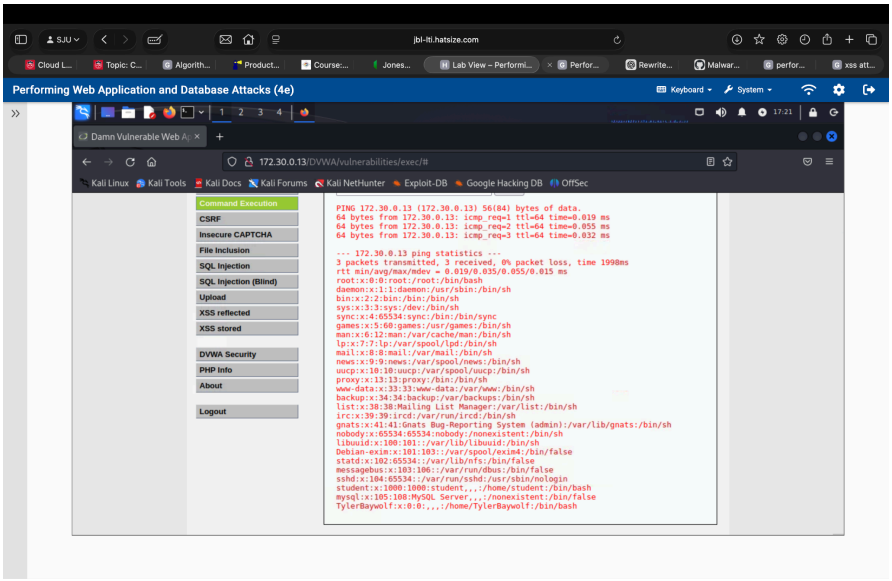


I input “a’ UNION ALL SELECT system_user() , user() ;#” to retrieve username and hostname. This returned root@localhost indicating that queries are executed through that user. The root user typically has access to everything on a SQL server. Next, I input “a’ UNION ALL SELECT user password FROM mysql.user;#”. This returned multiple records in the database including passwords and usernames.

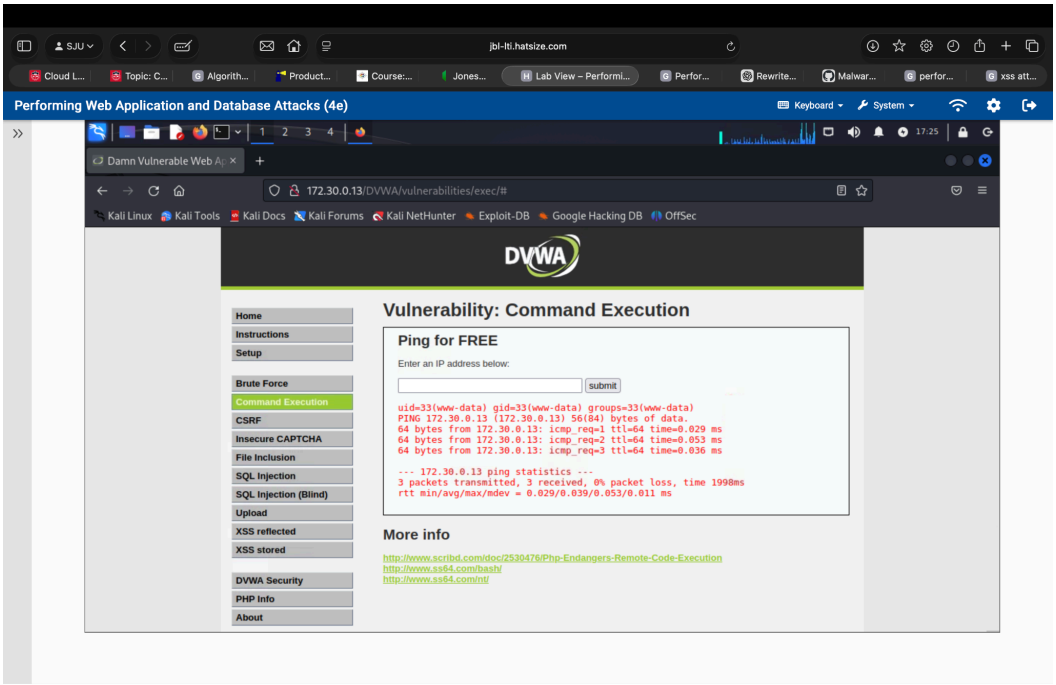


COMMAND EXECUTION

One of the biggest vulnerabilities is an application that contains execution of system commands. This type of vulnerability can allow any unauthorized and malicious user to execute commands from the web application to the web host system's operating system. I navigated to the command execution on the sidebar. In the Ping for Free field, I input the IP address “172.30.0.13:. The response returned from the PING request looks exactly like the response that would be returned by a command-line execution of a PING command. This means that the web server might be running the command through the host system’s operating system, and it opens the door for Command Execution Injections. Next, I input “172.30.0.13; cat /etc/passwd/ which returned a large list of records of usernames and passwords. This simple exploit demonstrates how a simple command execution can set an arbitrary command in a web server.

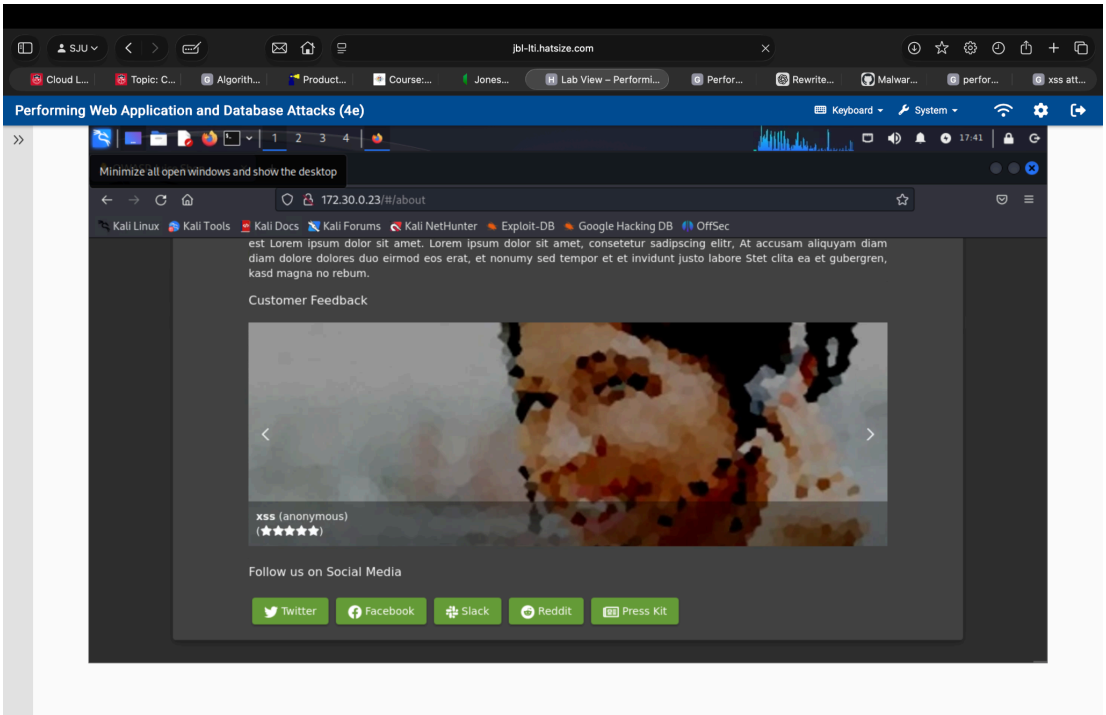


I input “172.30.0.13 & id” next. Since the ping command takes longer to complete, the id command finishes first and its result is displayed at the top. This contrasts with the previous example, which used a semicolon (;) to execute the commands sequentially.

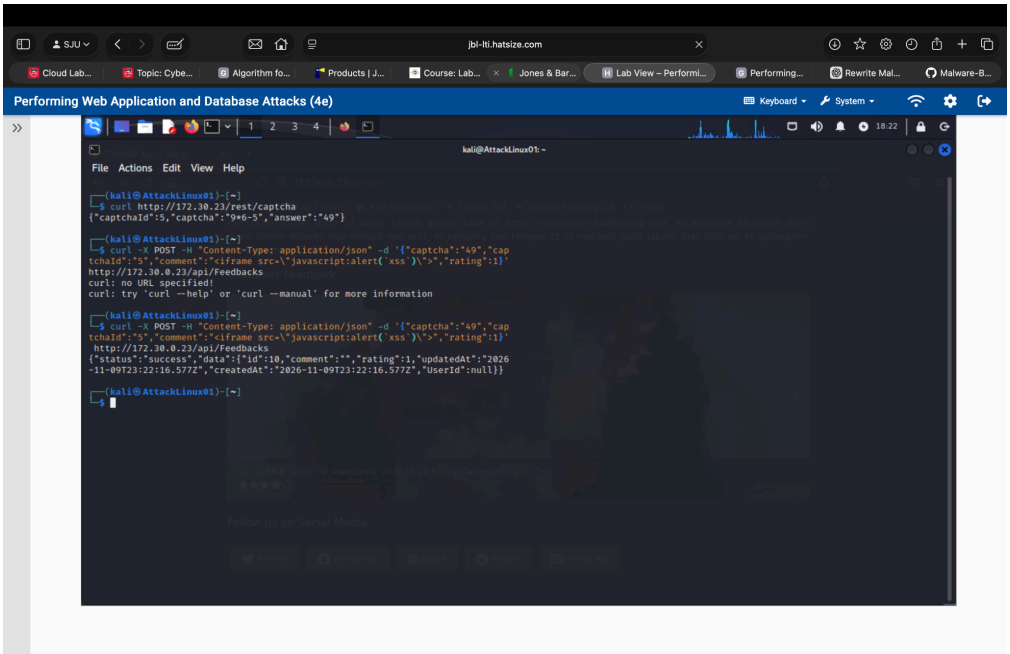


STORED XSS ATTACK ON OWASP JUICE SHOP

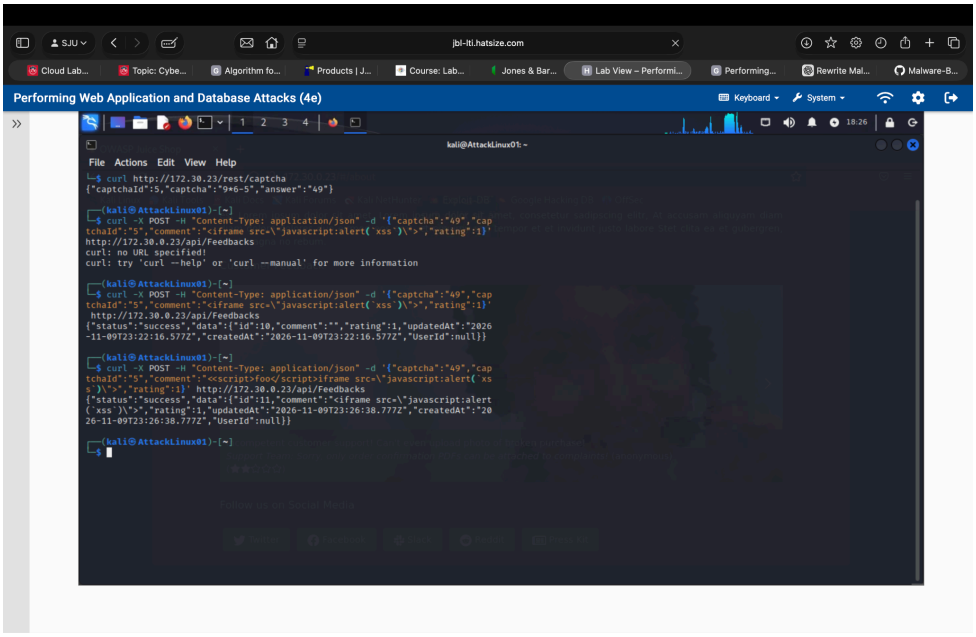
Next, I input the IP address of OWASP Juice Shop. This is a vulnerable wep application built with Javascript and is more realistic than DWVA. Real world web applications have multiple endpoints that can be potentially vulnerable. The IP address 172.30.0.23 was put in the search bar. Then, I went to the Customer Feedback on the sidebar. I created a comment that included the HTML “xss” and answered the CAPTCHA question. I went to the About Us section in the sidebar and found my comment there. This demonstrates that there isn’t input sanitization to protect against malicious user input.



Next, I used a curl command to get an JSON response returning strings for captchaid, the captcha answer, the rating given, and the comment to be added. I then ran another curl command to return a JSON object where the numbers substitute the answer and captchaid values. This command should get a successful result, which is a JSON object, but the comment field is an empty string rather than the iframe tag you tried to add. This indicates that there is some sanitization of the input happening.

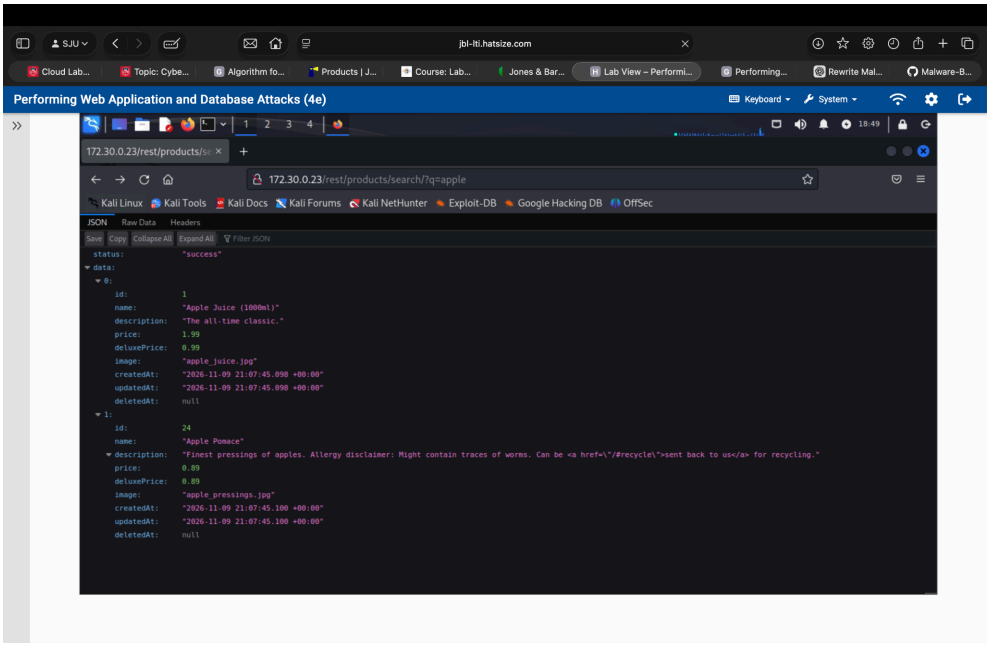


I used the same command but altered the comment with a new script. This returned a popup message when the browser was refreshed. Everytime, I returned to the browser the pop up would appear which demonstrates a successful stored XSS attack.

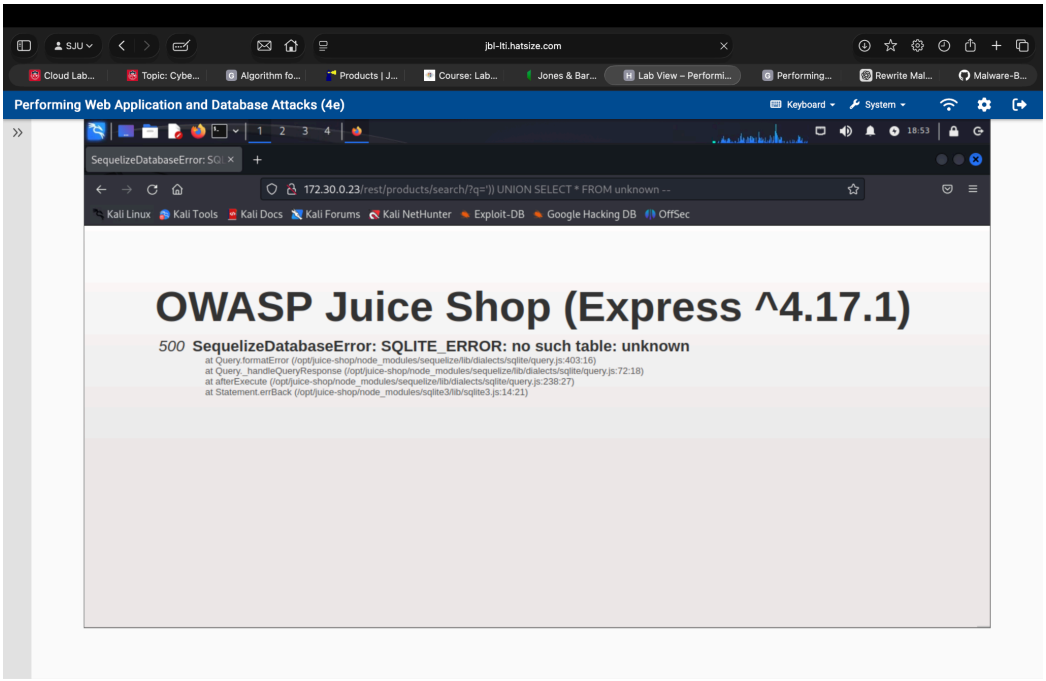


SQL INJECTION WITH OWASP JUICE SHOP

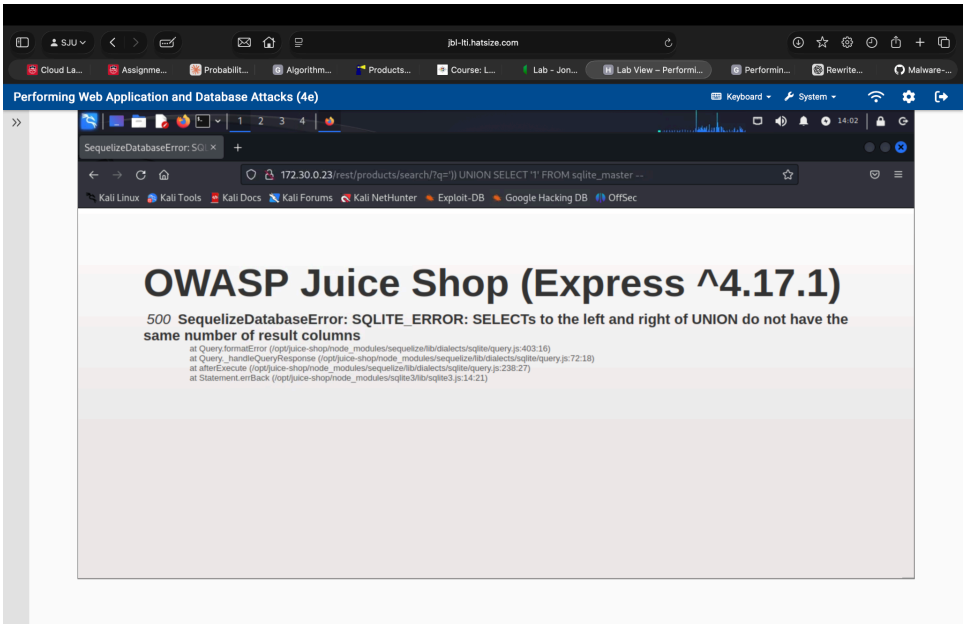
In the browser search, I entered http://172.30.0.23/rest/products/search/?q=apple. In the application, the page can send requests to this API in the background and then update the page with the results when they are received. A JSON object was returned displaying all information about the apple juice and pomace.



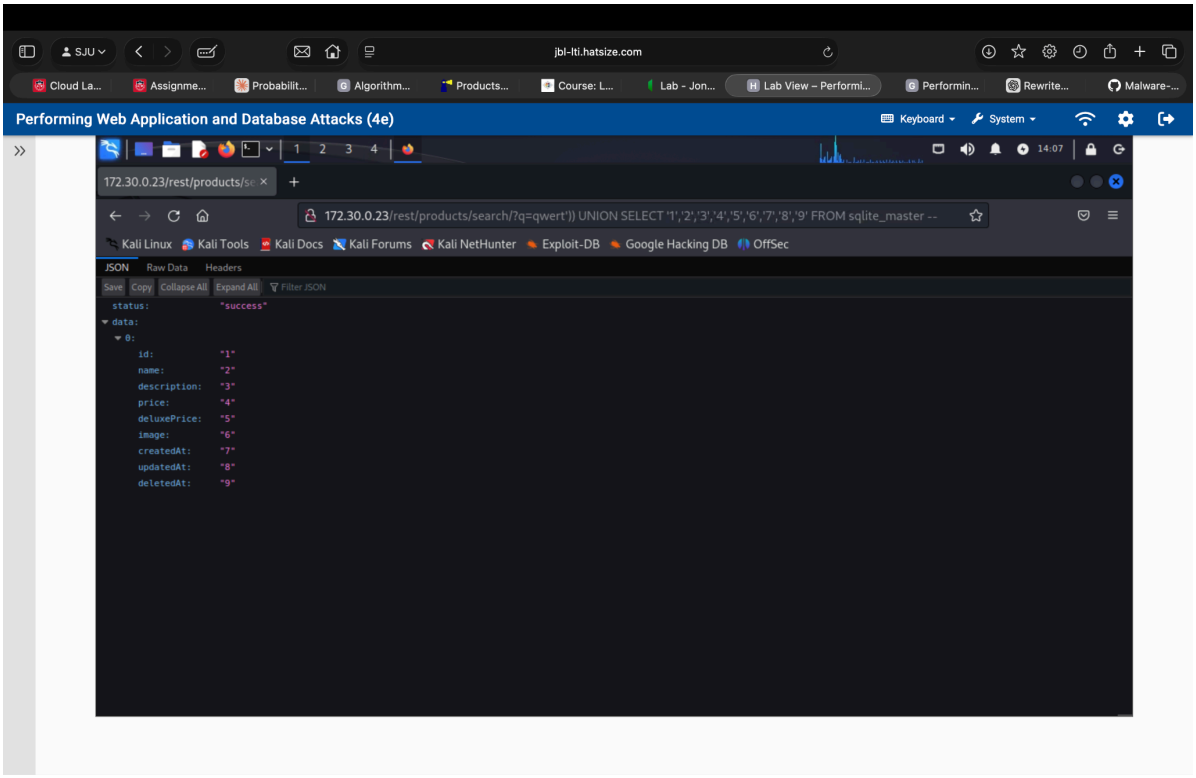
In the search bar, I changed the URL to "http://172.30.0.23/rest/products/search/?q=')) UNION SELECT FROM * unknown –" and received an error message stating there is no such table as unknown. During the process of an SQL injection many attempts are necessary in order to determine a structure that will work. It is common to use well known database table names to discover data.



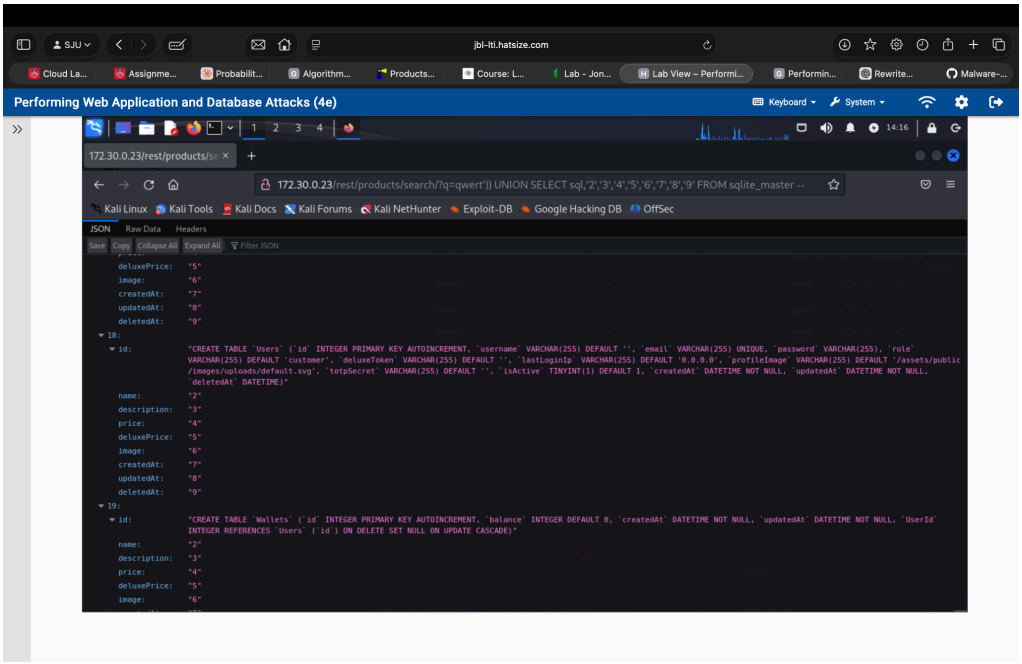
I changed the URL to http://172.30.0.23/rest/product/search/?q=')) UNION SELECT FROM * sqlite_master. Again it returned an error message however the error no longer says the table is unknown. It says that the parts of the union have different numbers of columns. The next step is to determine how many columns the unknown part of the query has. Next, I changed the "*" to '1' and this returned a new error message. This shows that the query has one column so the original query must have more columns.



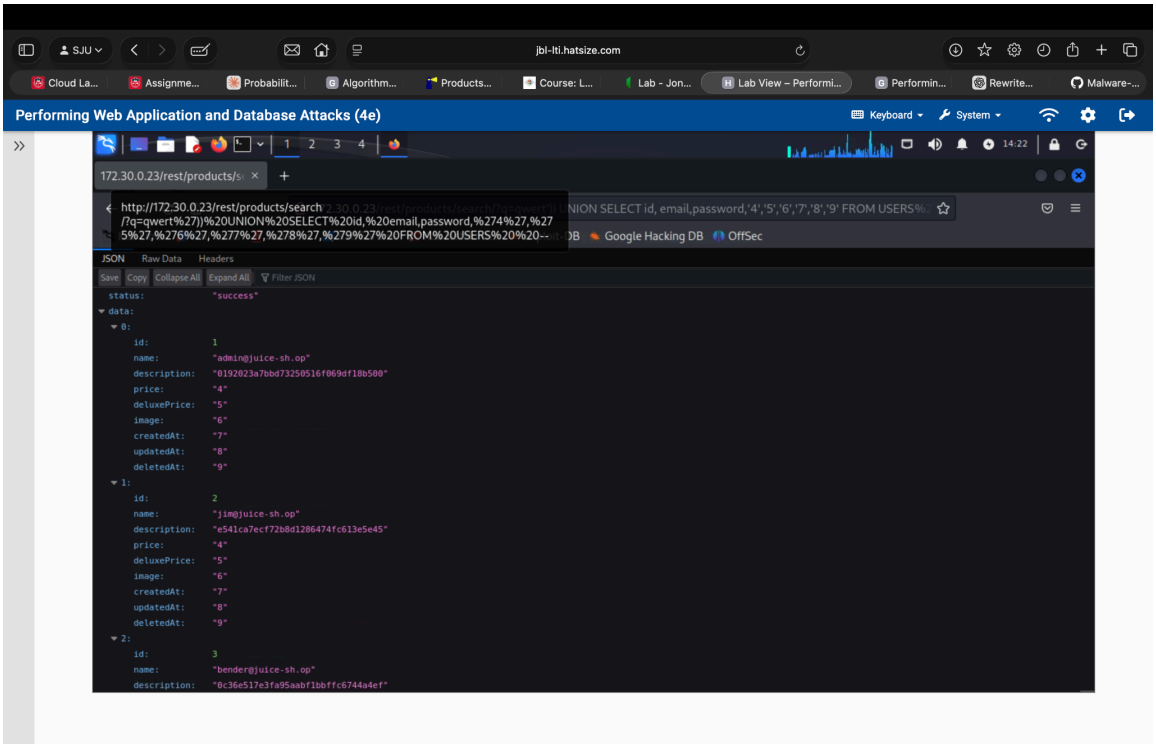
Next, I added '2', '3', '4', etc to return a JSON response displaying final entry values from 1 to 9. The original query for finding products is clouding the useful information. In the next query, I added "q=qwert". This removes noise from the results and returns the results from the query.



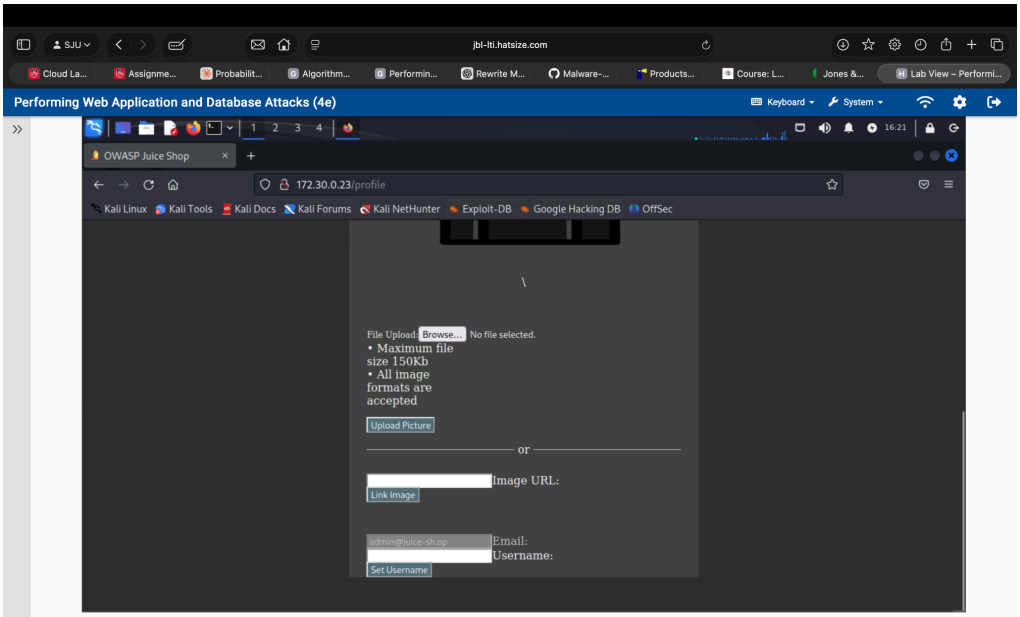
The next query, I changed the ‘1’ to “sql” and this returned many results. The sql_master column contains SQL statements to create the tables in the database. This provides information about the structure of the database itself. I searched for the users column of the JSON response to find the structure for how users are stored in the database.



From the previous screenshot, usernames, id, and passwords are stored inside the users table. With this information, I changed the URL again to search for this information in the table. I changed the URL to “http://172.30.0.23/rest/products/search/?q=qwert’)) UNION SELECT id, email, password, ‘4’,’5’,’6’,’7’,’8’,’9’ FROM USERS –” which retrieved key information from the database. I obtained emails, password hashes, and the user ID numbe for each account.

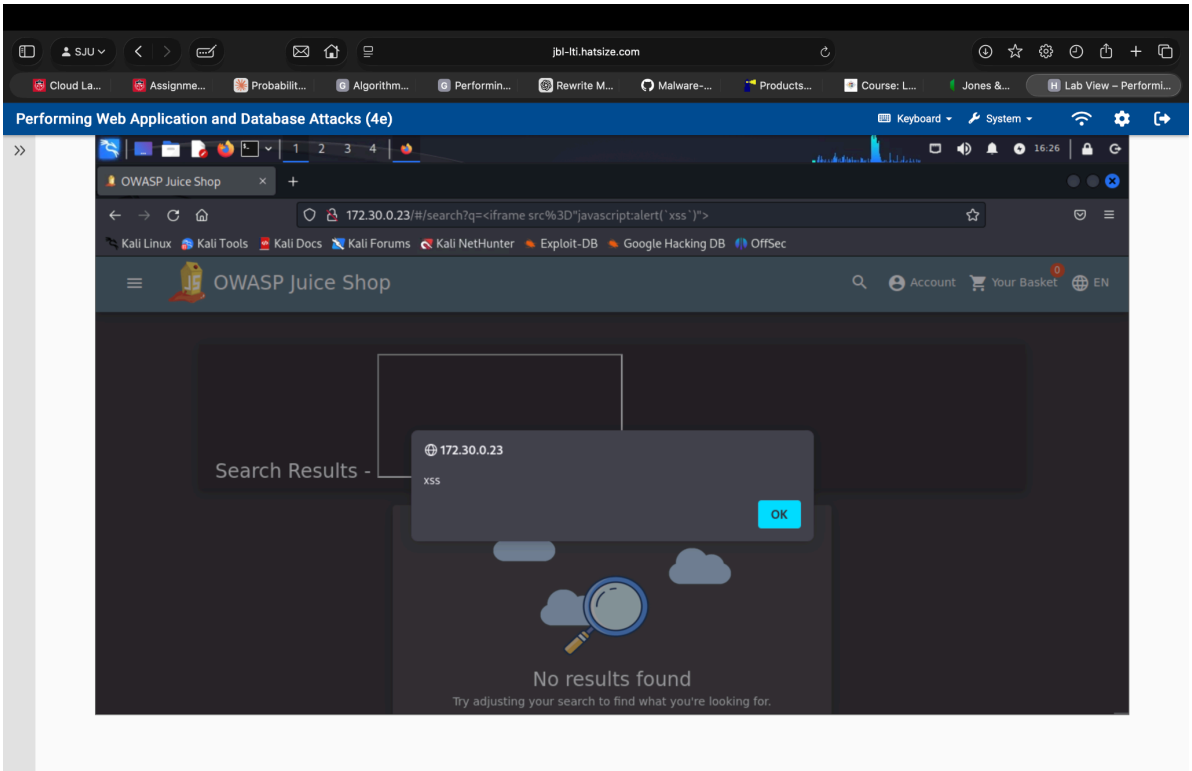


I went to the login page of 172.30.0.23 and input a new SQL statement of “ ‘ or 1=1–” and the password “anything”. The login check could be using a simple SQL statement to check the email and password.This gave access to the admin account of the website. The query condition is *email='' or 1=1*. Since 1 always equals 1, this condition is always true, and the login succeeds.



REFLECTED XSS ATTACK ON OWASP JUICE SHOP

For the final part of this lab, I constructed a Javascript URL to perform a reflected XSS attack on the Juice Shop website. In the URL, I added http://172.30.0.23/#/search?q=<iframe src="javascript:alert(`xss`)"> to the search bar. This performed the attack on the main menu of the shop with a message of “xss”.



Conclusion

This lab demonstrated how vulnerable web applications can be exploited using XSS attacks, SQL injection, and command execution. Working through DVWA and OWASP Juice Shop showed how each vulnerability stems from the same root issue: applications trusting user input instead of properly sanitizing and validating it before processing or displaying it.

The XSS attacks on DVWA showed the difference between reflected and stored attacks. A reflected attack only executes once when a crafted request is sent, while a stored attack persists and executes every time a victim loads the affected page, making it more dangerous since it doesn't require tricking a user into clicking a malicious link each time. The SQL injection section showed how small changes to an input field, like adding an apostrophe or using UNION SELECT, can reveal the structure of a database and eventually expose usernames and passwords. The command execution section showed that when a web application passes user input directly to the host system's command line, an attacker can chain additional commands using characters like semicolons or ampersands to run arbitrary code on the server.

Testing OWASP Juice Shop reinforced these same concepts in a more realistic environment. The stored XSS attack through the comment field showed that sanitization isn't always complete, since a payload that was blocked in one form still succeeded once the request was crafted and sent directly through curl. The SQL injection on the product search endpoint required multiple attempts to determine the number of columns and table names before the users table could be queried directly for emails and password hashes. The login bypass using "' or 1=1--" showed how a simple authentication check can be defeated when user input is inserted directly into a SQL query without parameterization. Finally, the reflected XSS attack using an iframe with a javascript: URL in the src attribute showed that filtering out script tags alone isn't enough, since other HTML elements can still be used to execute JavaScript in a victim's browser.

Overall, this lab reinforced why input validation, output encoding, parameterized queries, and proper error handling are necessary at every layer of a web application. Each vulnerability exploited in this lab could have been prevented with these basic security practices, which is why they are considered standard requirements in secure web development.